DOCUMENTAÇÃO FINAL

MOKKA

Um código para cada serviço. Sem repetição.

Nome do Projeto: MOKKA

Empresa: Gerdau

Programa: Desafio GC 2026 · Instituto Caldeira · Grupo 17

Equipe: Bárbara Baumgardt, Leonardo Lopes, Lukas Leimann, Thales Conci, Thirza Oliveira

Versão do sistema: v1.5 (6 de novembro de 2026)

Repositório: github.com/LeonardoLopesMMZ/DESAFIO-GC-GERDAU (branch main)

Sumário

1. Objetivo da documentação....3
2. Resumo....3
3. Problema de negócio....4
4. Estrutura da base de dados....5
5. Arquitetura da solução....5
6. Modelo de dados....10
7. Fluxo funcional....11
8. Regras de padronização....14
9. Critérios de similaridade....15
10. API e integrações....16
11. Instruções de uso....18
12. Instalação e configuração....21
13. Critérios de aceite....22
14. Tratamento de erros....23
15. Limitações....24
16. Segurança e preservação dos dados....25
17. Testes....25
18. Operação e manutenção....26
19. Solução de problemas....27
20. Evidências visuais....27
21. Próximas evoluções....29

1. Objetivo da documentação

Esta documentação explica a solução de organização dos códigos de serviço para que uma pessoa que não participou do desenvolvimento consiga compreender, instalar, usar e manter o projeto.

O documento apresenta:

- O problema que é resolvido.
- O objetivo da solução.
- Os dados utilizados.
- O funcionamento do processo.
- A arquitetura da solução.
- O papel de cada tecnologia.
- Os critérios de padronização.
- Os critérios de similaridade.
- As instruções de uso e de instalação.
- Os critérios de aceite e como foram atendidos.
- Os testes e as evidências.
- As limitações.
- Os pontos que dependem da Gerdau e as próximas evoluções.

2. Resumo

A empresa possui registros de serviços com descrições diferentes, abreviações, escalas, horários e formatos variados. Essa falta de padronização dificulta identificar quando duas descrições representam o mesmo serviço e provoca a criação de códigos duplicados.

O foco principal da solução é a coluna D (Descrição do Item). A partir da organização dessa coluna, o MOKKA facilita a consulta e a relação com a coluna C (Código do Serviço).

A solução:

1. Preserva os dados originais.
2. Padroniza as descrições da coluna D.
3. Localiza descrições iguais ou semelhantes.
4. Apresenta sugestões ao usuário.
5. Permite a reutilização de um código existente.
6. Permite a solicitação de um novo código quando necessário.
7. Registra validações, aprovações e alterações.

Resultados na base real da Gerdau:

Indicador	Resultado
Códigos na base analisada	7.937
Códigos repetidos encontrados	883 (11,1%, 1 em cada 9)
Catálogo final	7.054 códigos oficiais; cada repetido aponta para o oficial

Indicador	Resultado
Serviços diferentes juntados por engano	0 em 430 pares conferidos à mão
Erros de digitação e de fala resolvidos	89% (296 de 334)
Nota de saúde do catálogo	77 → 100
Tempo por consulta	menos de 10 milissegundos
Planilha em lote	até 100.000 linhas; a base inteira em cerca de 7 segundos
Teste geral de ponta a ponta	35 de 35

3. Problema de negócio

3.1 Problema principal

As descrições dos serviços não seguem um padrão único. Um mesmo serviço pode ser registrado com abreviações, escalas, horários e formatos diferentes. Como consequência, fica difícil identificar duplicidades e escolher o código de serviço correto.

O problema pode ser resumido como: *como organizar a descrição dos serviços para facilitar a identificação, a reutilização e o controle dos códigos existentes?*

Exemplo real: o mesmo encarregado em hora extra de segunda a sábado aparece na base como "ENCARREGADO HE SEG-SAB", "SV ENC HE SAB" e mais seis formas, cada uma com um código. Isso acontece porque cada comprador escreve de um jeito e o SAP aceita só 40 caracteres na descrição.

3.2 Consequências do problema

A desorganização causa:

- Criação de códigos duplicados.
- Dificuldade para localizar serviços.
- Escolha incorreta de códigos.
- Retrabalho para as equipes.
- Informações de custos distribuídas em registros diferentes.
- Relatórios inconsistentes.
- Dificuldade para comparar fornecedores.
- Perda de histórico.
- Baixa confiabilidade da base.

3.3 Categorias do piloto

As categorias escolhidas são Eletromecânica, Facilities e Civil. Elas possuem grande quantidade de fornecedores e variedade de serviços, sendo adequadas para testar a solução.

Grupo	Participação nos repetidos
Eletromecânica	19% dos códigos do grupo eram repetidos
Civil	9%
Facilities	8%

4. Estrutura da base de dados

Coluna	Campo	Função
A	Grupo de Serviço	Indica a categoria do serviço
B	Código do Fornecedor	Identifica o fornecedor
C	Código do Serviço	Identifica o serviço utilizado pela empresa
D	Descrição do Item	Descreve o serviço e é o principal campo de padronização
E	Unidade de medida	Indica a unidade utilizada, como SV, HH ou H

4.1 Coluna D: Descrição do Item

A coluna D é o foco do projeto. Ela pode conter abreviações diferentes; espaços, hífens e barras usados de forma inconsistente; escalas e horários escritos de maneiras diferentes; descrições incompletas; textos truncados em 40 caracteres e informações misturadas sem estrutura fixa.

4.2 Coluna C: Código do Serviço

A coluna C contém o código associado ao serviço. O MOKKA facilita a consulta deste código a partir da descrição padronizada.

O código nunca é alterado automaticamente só porque duas descrições são parecidas. Casos duvidosos vão para o curador, a pessoa responsável por validar.

4.3 Dados brutos

Os dados brutos são os dados recebidos originalmente da planilha da Gerdau. Eles são preservados sem alterações.

Local dos dados brutos: tabela base_original no banco (linha por linha, como veio) e a planilha original na pasta saneamento/dados do computador do grupo. A planilha é confidencial e não vai para o GitHub; para quem não tem acesso a ela, o repositório traz uma base fictícia no mesmo formato (saneamento/exemplo/base_exemplo.csv).

4.4 Dados padronizados

Os dados padronizados são os registros tratados pela solução e mantêm a relação com os dados originais: cada código guarda a descrição original, o texto limpo, o nome padrão de até 40 caracteres e, quando é repetido, o código oficial para o qual aponta.

Local dos dados padronizados: tabela codigo_servico.

5. Arquitetura da solução

5.1 Visão geral

A solução é composta por quatro partes principais:

1. **Front-end:** interface utilizada pelo usuário.
2. **Java:** API, regras de negócio e segurança.
3. **Python:** importação, limpeza, similaridade e análise dos dados.
4. **PostgreSQL:** armazenamento dos dados, códigos, validações e histórico.

Fluxo geral:

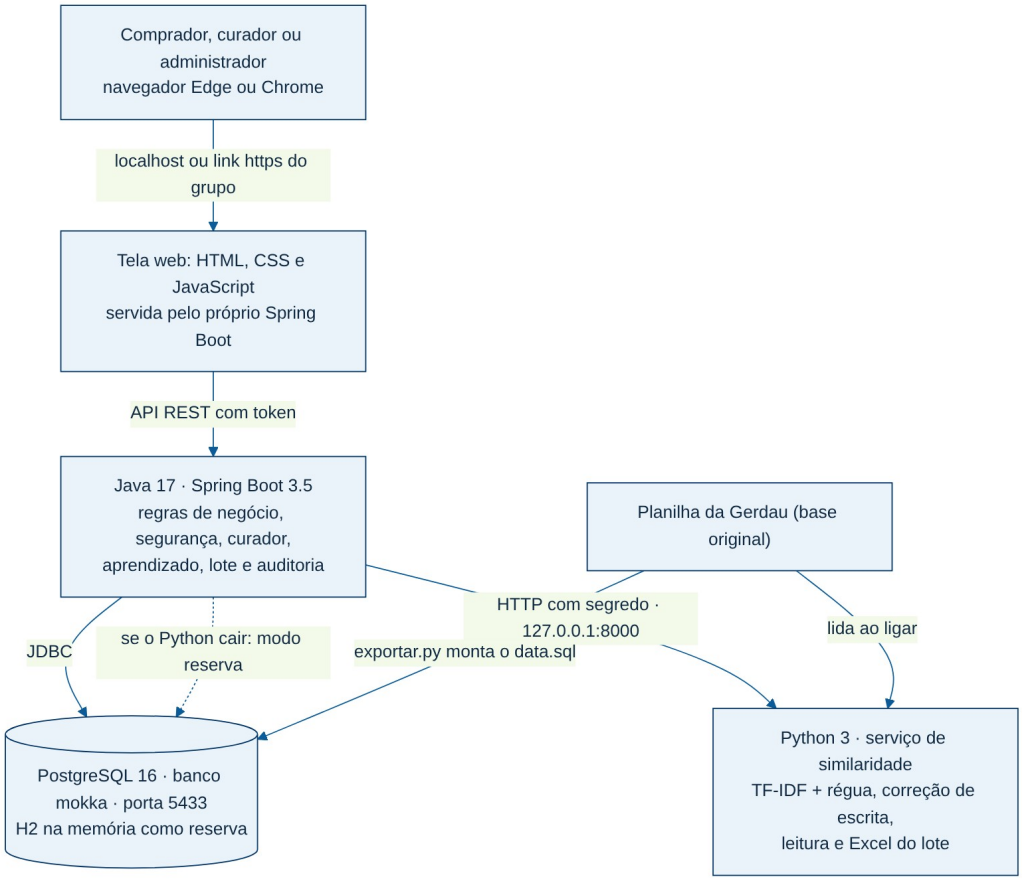


Figura 1. Arquitetura do MOKKA e como as partes conversam

5.2 Arquitetura real do projeto

O repositório está organizado assim (a lista completa de arquivos está no GitHub):

Pasta ou arquivo	Conteúdo
MOKKA.bat / MOKKA.command	liga tudo no Windows, no Mac e no Linux: prepara o Python, compila o Java, liga o PostgreSQL e abre o site
saneamento/	Python: padronização, deduplicação, similaridade, correção de escrita, lote, Excel, serviço HTTP, validação e testes
saneamento/docs/	Entrega 1 e Entrega 2 (etapas 01 a 08, dados)
saneamento/exemplo/	base fictícia no formato da Gerdau
codigos-api/	Java e Spring Boot: API, regras, curador, aprendizado, segurança e a tela (src/main/resources/static)
docs/09 a docs/14	Entrega 2 (Java e banco)
docs/15 a docs/23	Entrega 3
docs/24 a docs/28	Entrega final
docs/COMO_TESTAR.md, COMO_ABRIR_SEM_BAT.md	roteiro de teste à mão e como ligar sem o .bat
scripts/	teste geral (testar.ps1, 35 testes) e PostgreSQL portátil

5.3 Front-end

O front-end é a parte visual da aplicação, aberta no navegador. Ele permite:

- Consultar serviços por texto ou por voz.

- Pesquisar por código no catálogo e ver a ficha de cada código.
- Visualizar dados brutos e padronizados lado a lado.
- Analisar as sugestões, de 5 em 5, com o que muda em cada uma.
- Confirmar uma correspondência (usar o código).
- Corrigir uma palavra sugerida pelo MOKKA.
- Solicitar um novo código, com justificativa.
- Analisar uma planilha inteira (lote).
- Decidir os pedidos (curador).
- Acompanhar tudo no painel Raio-X (curador e administrador).
- Aprender a usar pelo Tutorial.

Tecnologias: HTML, CSS e JavaScript, sem framework, servidos pelo próprio Spring Boot (não usamos Bootstrap). Recursos do navegador: reconhecimento de voz (Web Speech), câmera para a entrada pelo rosto (face-api.js), chave do Windows (WebAuthn) e modo local (service worker).

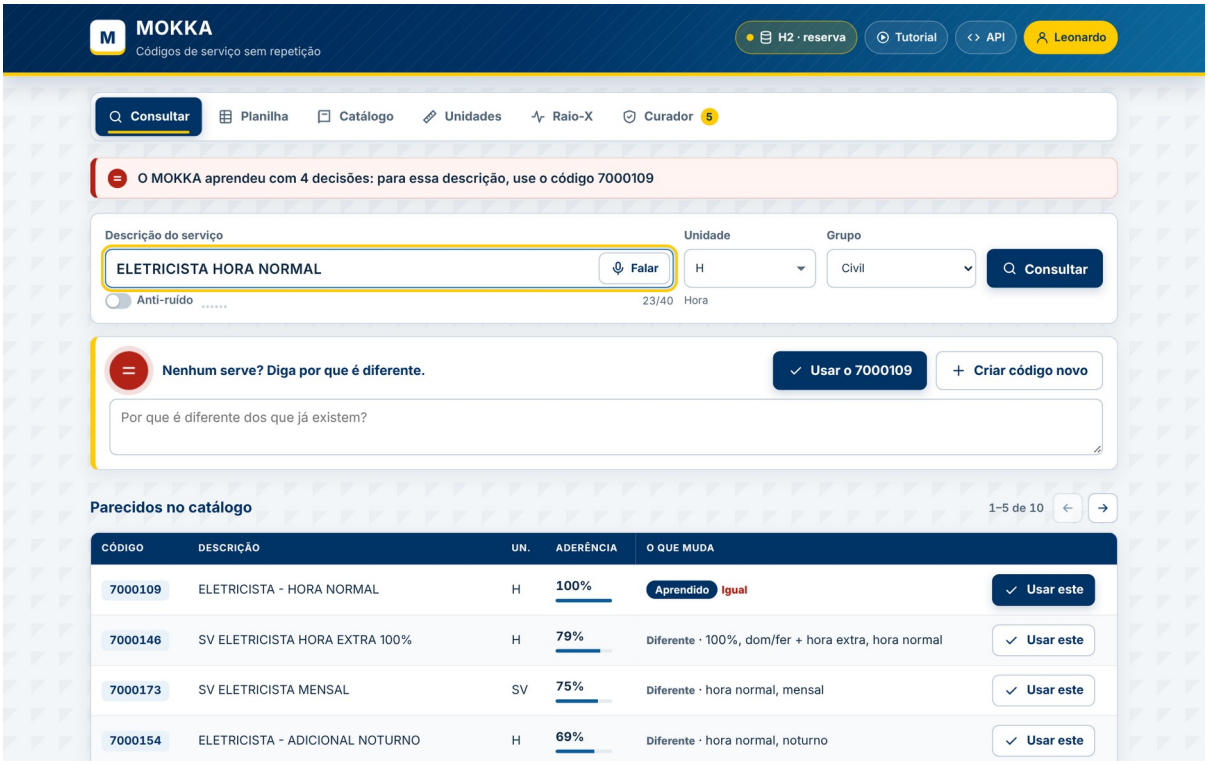


Figura 2. Tela de consulta

5.4 Java e API (Application Programming Interface)

O Java é responsável pela comunicação entre o front-end, o Python e o PostgreSQL, além de controlar as regras de negócio.

Responsabilidades:

- Receber consultas e devolver o resultado com os candidatos.
- Consultar e gravar dados no banco.
- Validar informações (tamanho, unidade, perfil).
- Receber solicitações de novos códigos.
- Controlar aprovações e rejeições do curador.

- Aprender com as decisões.
- Registrar histórico e auditoria.
- Controlar a entrada (rosto, chave do Windows, nome) e os perfis.
- Retornar mensagens de erro em português.

Tecnologias: Java 17, Spring Boot 3.5, API REST, Spring JDBC, validação (Bean Validation), Swagger (springdoc) em /swagger.

Pacote	O que faz
consulta	consulta, chamada ao Python e modo reserva pelo banco
solicitacao	decisão do comprador, fila e decisões do curador
aprendizado	regras IGUAL e DIFERENTE e palavras ensinadas
catalogo	catálogo, ficha do código, unidades e Raio-X
lote	planilha: analisar, importar e exportar
acesso	entrada pelo rosto, chave do Windows e nome; perfis; fotos criptografadas
sistema	segurança (Guarda), auditoria, administração e carga do catálogo

5.5 Python e dados

O Python é responsável pelo tratamento e pela análise dos dados.

Responsabilidades:

- Importar a base original.
- Validar os campos.
- Preservar os dados brutos.
- Normalizar a coluna D.
- Comparar descrições.
- Identificar possíveis duplicidades.
- Sugerir o nome padrão e a categoria.
- Corrigir erros de escrita e de fala.
- Ler planilhas, CSV, texto e JSON e gerar o Excel organizado.
- Gerar relatórios e a validação.

Scripts utilizados:

Script	Função
diagnostico.py	diagnóstico de qualidade e análise inicial da base: vazios, duplicados, unidades, tamanhos
padronizar.py	padronização do texto em 5 níveis, expansão de abreviações (dicionário de 136 termos) e leitura dos atributos
limpar.py	nome padrão de até 40 caracteres e categoria de cada serviço
regua.py	régua de similaridade: regras que dizem se dois serviços são iguais, parecidos ou diferentes
deduplicar.py	junta os códigos repetidos e escolhe o código oficial de cada grupo
exportar.py	gera as tabelas do banco (data.sql) para o H2 ou o PostgreSQL
similaridade.py	motor de busca (TF-IDF): acha os parecidos, corrige escrita e analisa o lote

Script	Função
organizar_planilha.py	arruma planilhas bagunçadas: títulos, abas, cabeçalhos repetidos, células vazias
lote.py	lê o lote (Excel, CSV, texto ou JSON, até 100.000 linhas) e gera o Excel de saída
api.py	serviço que o Java chama (127.0.0.1:8000): consulta, correção, lote e Excel
validar.py	mede a similaridade: 430 pares conferidos, erros de escrita e sem correspondência
test_padronizar.py	testes automáticos (9 testes)

Tecnologias: Python 3.10 a 3.13, pandas, openpyxl, numpy, scikit-learn, SQLAlchemy e psycopg.

5.6 PostgreSQL

O PostgreSQL armazena os dados brutos, os dados padronizados, os códigos de serviço, os códigos de fornecedores, os grupos, as unidades, as sugestões, as solicitações, as validações e o histórico.

Nome do banco: mokka

Ambiente: desenvolvimento. PostgreSQL 16 portátil, no próprio computador de quem liga (127.0.0.1, porta 5433, usuário mokka_app). Sem o PostgreSQL, o sistema usa o H2 na memória, com o mesmo esquema.

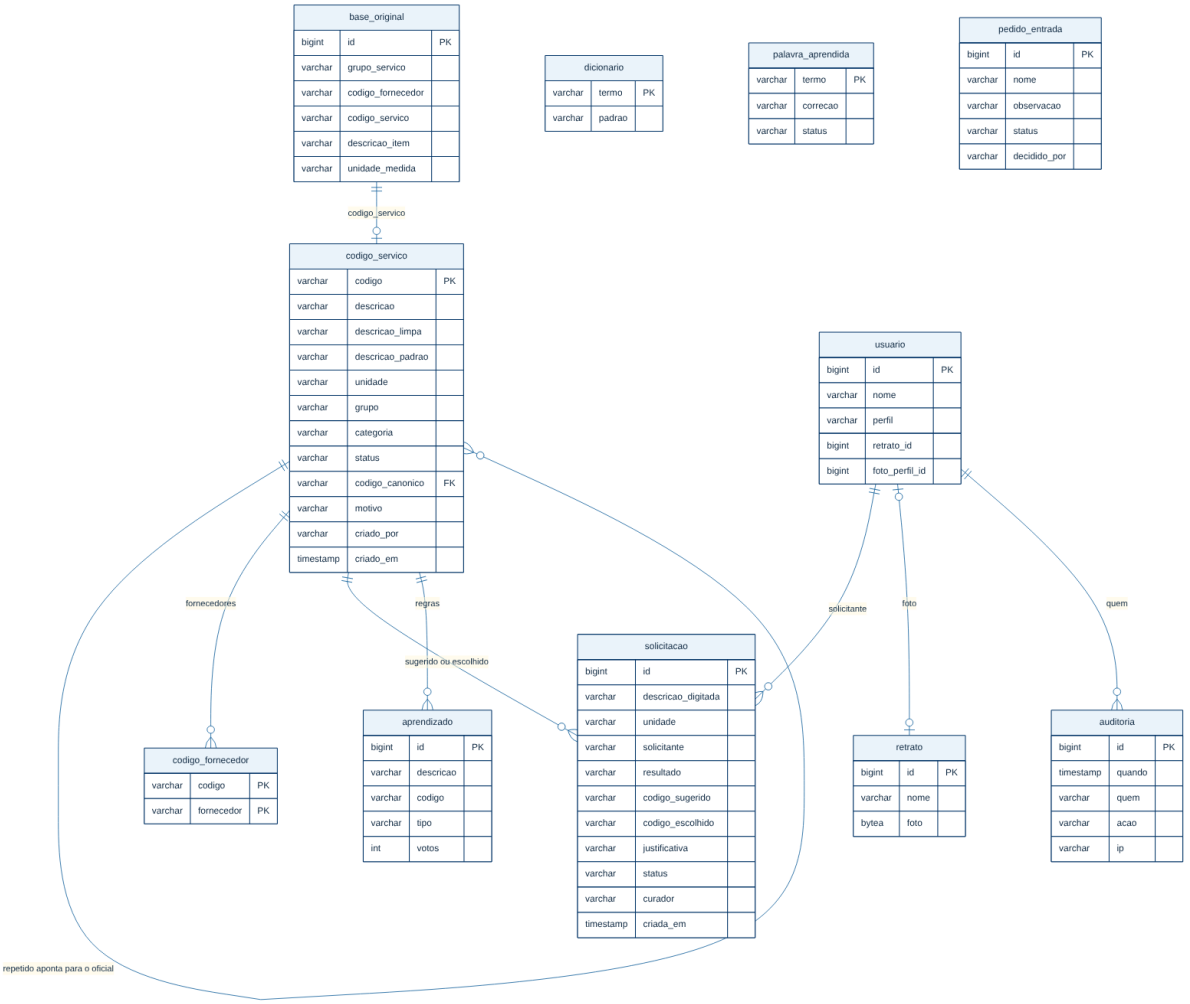


Figura 3. Modelo das tabelas (schema.sql)

6. Modelo de dados

6.1 Dados brutos

Tabela base_original: armazena os registros exatamente como foram recebidos.

Campo	Conteúdo
id	identificador
grupo_servico	grupo de serviço (coluna A)
codigo_fornecedor	código do fornecedor (coluna B)
codigo_servico	código do serviço (coluna C)
descricao_item	descrição original (coluna D)
unidade_medida	unidade de medida (coluna E)

A data de importação é a da carga: o catálogo é montado de novo a partir da planilha toda vez que o MOKKA liga, sem passo manual.

6.2 Dados padronizados

Tabela codigo_servico: o resultado do tratamento de cada código.

Campo	Conteúdo
codigo	código do serviço (chave)
descricao	descrição original
descricao_limpa	texto padronizado usado na comparação
descricao_padrao	nome padrão de até 40 caracteres
unidade, grupo, categoria	classificação
status	ATIVO (oficial) ou BLOQUEADO (repetido)
codigo_canonico	referência ao código oficial, quando é repetido
motivo	por que foi juntado, criado ou importado
criado_por, criado_em	quem criou e quando (códigos novos)

6.3 Relacionamento de código

A tabela codigo_fornecedor liga cada código aos seus fornecedores. O campo codigo_canonico liga cada código repetido ao código oficial, e a tabela dicionario guarda os 136 termos de abreviação.

6.4 Solicitações

A tabela solicitacao controla cada consulta e cada pedido de código novo: descrição digitada, unidade, grupo, solicitante, como entrou, resultado (BLOQUEAR, REVISAR ou LIBERAR), código sugerido, candidatos mostrados, código escolhido, justificativa, status (PENDENTE, USOU_EXISTENTE, CRIADO, AGUARDANDO_CURADOR, RECUSADO), curador e datas.

6.5 Histórico

Tabela	O que registra
auditoria	quem fez o quê, quando e de onde: entradas, códigos criados, decisões, desfazer, perfis, fotos apagadas
aprendizado	regras IGUAL e DIFERENTE aprendidas com as decisões, com os votos
palavra_aprendida	palavras corrigidas pelos compradores, que só valem depois do curador
pedido_entrada	pedidos de entrada sem câmera e quem liberou
usuario, retrato	pessoas, perfis e fotos de entrada (criptografadas)

7. Fluxo funcional

7.1 Consulta

1. O usuário informa uma descrição, por texto ou por voz, com a unidade e o grupo.
2. O front-end envia a consulta para a API Java.
3. O Java corrige a escrita e abrevia para 40 caracteres, se for preciso.
4. O Java pede ao Python os candidatos (similaridade e régua) e aplica o que o MOKKA já aprendeu.
5. Os candidatos são organizados por relevância.
6. O usuário vê o resultado (já existe, parecido ou novo) e os 5 parecidos mais próximos; as setas mostram os próximos.

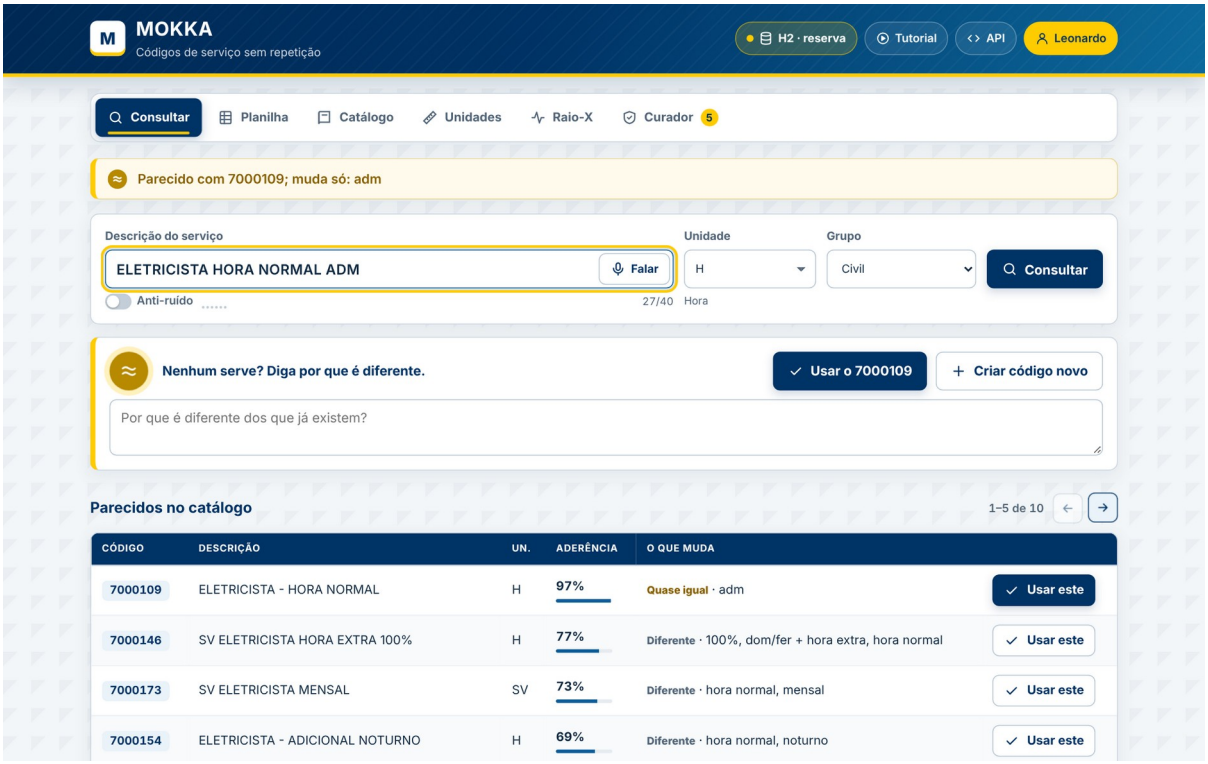


Figura 4. Consulta com resultado "parecido": o MOKKA mostra o que muda

7.2 Reutilização de código

Quando um código adequado é encontrado, o usuário confere a descrição original e a padronizada, a unidade e o que muda, e clica em "Usar este". A decisão é registrada e vira um voto para o MOKKA aprender.

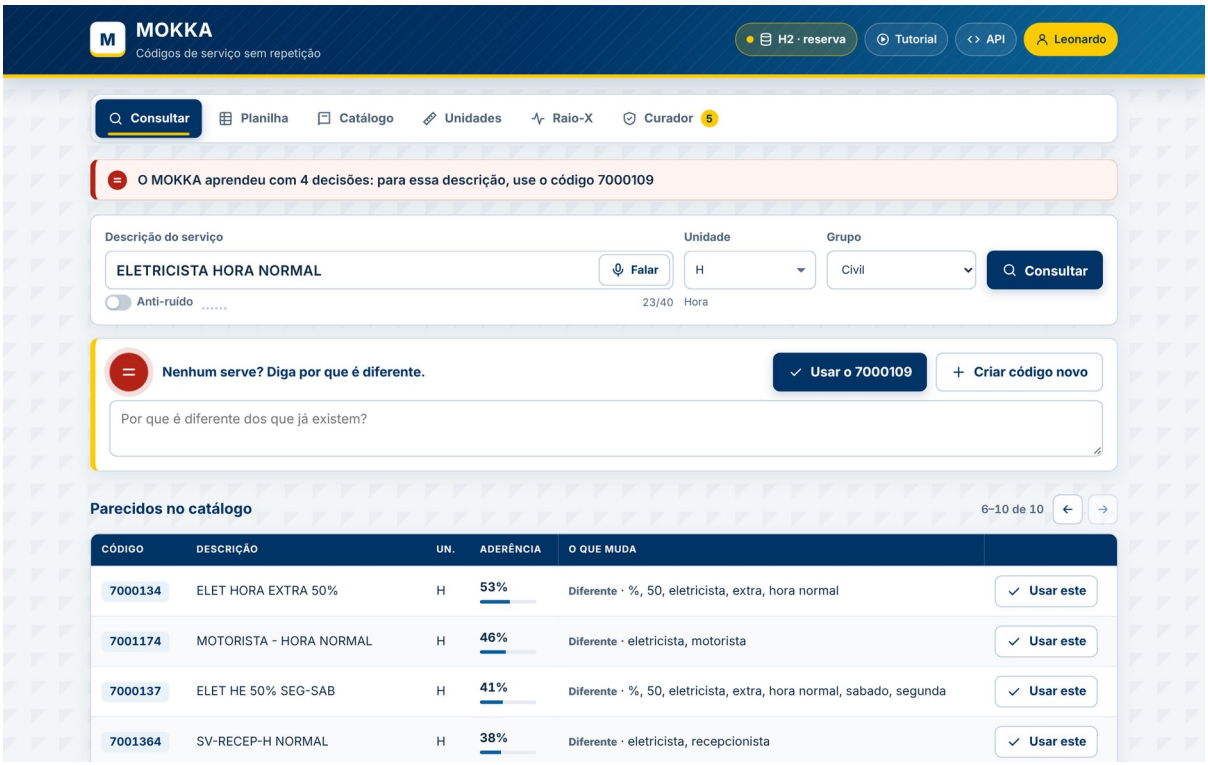


Figura 5. Lista de parecidos, de 5 em 5, com o botão Usar este

7.3 Solicitação de novo código

1. O usuário informa a descrição, a unidade e o grupo.
2. O MOKKA verifica possíveis duplicidades.
3. Se não há nada igual (LIBERAR), o código é criado na hora, com o nome padrão, e já entra na busca.
4. Se há um parecido (REVISAR), o usuário precisa escrever por que é diferente.
5. Se já existe (BLOQUEAR) e o usuário insiste, o pedido vai para o curador.
6. O curador vê os pedidos iguais juntos, com a sugestão e a certeza do MOKKA, e aprova ou recusa (com Desfazer).
7. Se aprovado, o código novo é criado no catálogo; se recusado, o pedido usa o código que já existia.

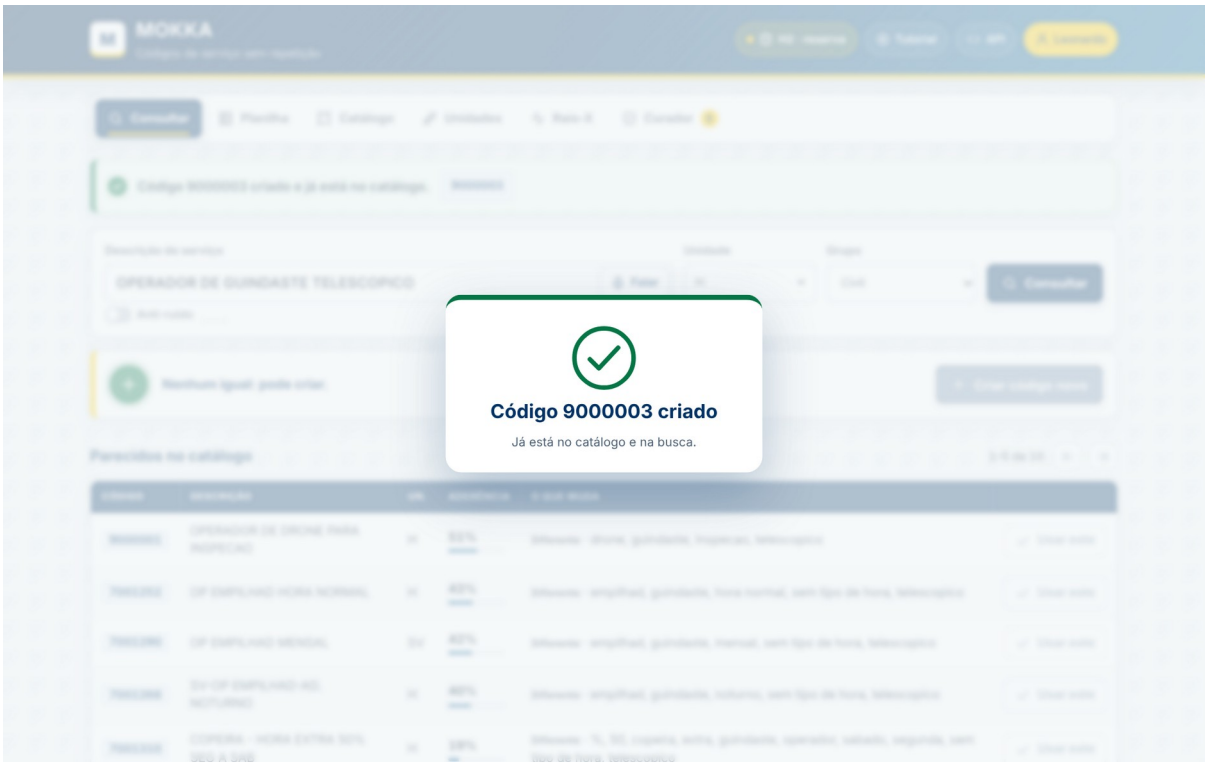


Figura 6. Código novo criado na hora

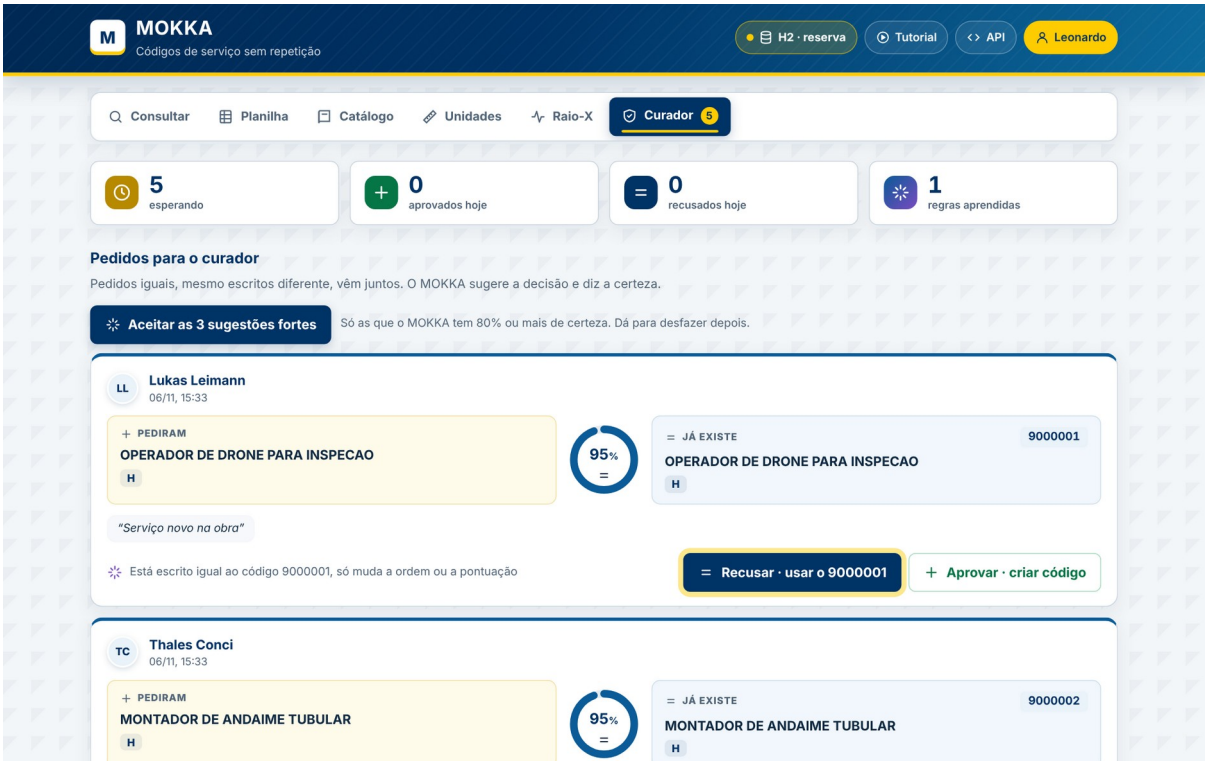


Figura 7. Curador: pedidos iguais juntos, sugestão e certeza

7.4 Fluxo resumido

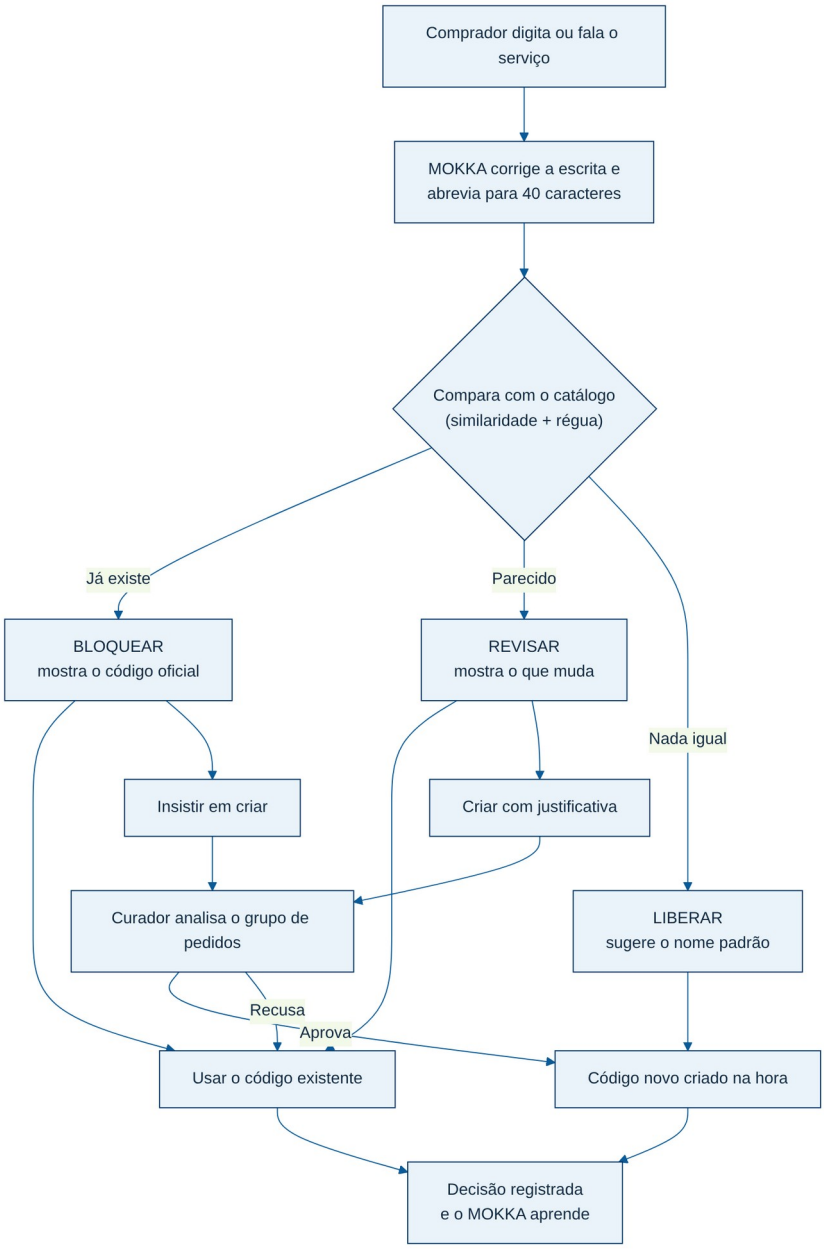


Figura 8. Fluxo do usuário

8. Regras de padronização

As descrições devem ser claras, consistentes e comparáveis.

8.1 Regras gerais

- Utilizar um padrão único de escrita.
- Padronizar letras maiúsculas e minúsculas.
- Remover espaços duplicados.
- Padronizar hífens e barras.
- Padronizar abreviações.
- Padronizar escalas.

- Padronizar horários.
- Remover termos desnecessários (como "SV" ou traço na frente).
- Respeitar o limite de 40 caracteres.
- Não colocar preço na descrição.
- Não colocar o nome do fornecedor na descrição.
- Preservar sempre o texto original.

8.2 Como o MOKKA aplica as regras

- **5 níveis de limpeza:** maiúsculas e acentos, pontuação, espaços, abreviações e forma canônica para comparar.
- **Dicionário:** 136 termos tirados da própria base (HN → HORA NORMAL, HE → HORA EXTRA, ENC → ENCARREGADO...).
- **Nome padrão:** até 40 caracteres, abreviando no padrão que a base já usa (ex.: ENCARREGADO DE MANUTENÇÃO MECÂNICA HORA EXTRA 50% SEGUNDA A SÁBADO vira ENCARREG MANUT MECÂNICA HE 50% SEG A SAB).
- **Atributos lidos:** cargo, tipo de hora, percentual, escala, medidas e unidade.

9. Critérios de similaridade

9.1 Campos analisados

- Descrição original e descrição padronizada (TF-IDF com pedaços de 2 a 4 letras).
- Unidade de medida (unidade diferente nunca é o mesmo serviço).
- Números da descrição: percentual, medidas, quantidades.
- Função ou cargo.
- Escala e horário (hora normal, hora extra, noturno, sábado, domingo).
- Grupo de serviço, para filtrar a busca.
- Decisões anteriores (o que o MOKKA aprendeu).

O código do fornecedor não entra na comparação: o mesmo serviço pode ter vários fornecedores, e todos ficam ligados ao código oficial.

9.2 Classificação dos resultados

Classificação	Significado	No MOKKA
Correspondência forte	Os dados indicam que os serviços são iguais	DUPLICADO → BLOQUEAR: mostra o código oficial
Correspondência possível	Existem semelhanças, mas é necessária confirmação	QUASE → REVISAR: mostra o que muda e pede justificativa
Serviços relacionados	Parecidos, mas devem continuar separados	DIFERENTE: aparece na lista, com o que muda
Sem correspondência	Nenhum registro adequado foi encontrado	LIBERAR: pode criar, com o nome padrão
Duplicidade confirmada	A área responsável confirmou que são o mesmo serviço	decisão do curador ou do comprador, que vira regra aprendida

Texto parecido não é serviço igual: "HE 40%" e "HE 50%" têm 91% do texto em comum e continuam separados. Por isso a estatística (TF-IDF) acha os candidatos e a régua de regras decide.

9.3 Ranking

Os candidatos são ordenados pela classe (igual, quase igual, diferente), depois pela aderência do texto e, no empate, pelo código. O que o MOKKA aprendeu vem primeiro, com "aprendido com N decisões".

#	Código	Descrição	Un.	Aderência	Classe	O que muda
1	7000109	ELETRICISTA - HORA NORMAL	H	100%	DUPLICADO	igual (aprendido com 4 decisões)
2	7000146	SV ELETRICISTA HORA EXTRA 100%	H	79%	DIFERENTE	100%, dom/fer, hora extra
3	7000173	SV ELETRICISTA MENSAL	SV	75%	DIFERENTE	hora normal, mensal
4	7000154	ELETRICISTA - ADICIONAL NOTURNO	H	69%	DIFERENTE	hora normal, noturno

Exemplo real retornado para "ELETRICISTA HORA NORMAL", unidade H, com a base de exemplo do repositório.

10. API e integrações

10.1 Endpoints

Todas as rotas, com exemplos, ficam no Swagger em http://localhost:8080/swagger. Tudo exige o token de entrada, menos a entrada em si.

Método	Rota	Função
POST	/api/acesso/entrar	entra pelo nome
POST	/api/acesso/rosto/cadastrar · /rosto/entrar	cadastro e entrada pelo rosto
POST	/api/acesso/chave/desafio · /chave/cadastrar · /chave/entrar	cadastro e entrada pela chave do Windows
POST	/api/acesso/pedir-entrada · GET /pedir-entrada/{id}	entrada sem câmera, liberada por um curador
GET / POST	/api/acesso/pedidos-de-entrada · /{id}/decisao	curador vê e libera os pedidos de entrada
GET	/api/acesso/eu	dono do token
POST / DELETE	/api/acesso/foto-perfil · DELETE /api/acesso/rosto	foto do perfil; apagar o próprio rosto (LGPD)
POST	/api/consultas	consulta uma descrição; devolve o resultado, os candidatos e a correção sugerida
POST	/api/consultas/palavras	confere palavras para o corretor
POST	/api/solicitacoes/{id}/decisao	usar um código existente ou criar um novo
GET	/api/solicitacoes	histórico de consultas e decisões
GET	/api/solicitacoes/curador · /curador/resumo	pedidos do curador agrupados e números do dia
POST	/api/solicitacoes/{id}/curadoria · /curador/grupo · /curador/desfazer	o curador aprova, recusa (um ou o grupo) e desfaz
GET / POST / DELETE	/api/aprendizado · /palavras · /regras/{id}	o que o MOKKA aprendeu; palavras ensinadas

Método	Rota	Função
POST	/api/lotes/analisar · /importar · /exportar	planilha: analisar, gravar no banco e baixar o Excel
GET	/api/codigos?busca= · /resumo · /padronizados · /novos · /{codigo}	catálogo, totais, oficiais com repetidos, criados e ficha
GET	/api/unidades	unidades, sinônimos e exemplos
GET	/api/raio-x · /api/raio-x/painel	saúde do catálogo e painel de tudo o que acontece (curador)
GET / POST	/api/admin/pessoas · /auditoria · /desbloquear	administração (perfil ADMIN)
GET	/api/sistema	banco em uso e situação do sistema

10.2 Exemplo de requisição

```
POST /api/consultas
Authorization: Bearer <token>
Content-Type: application/json

{
  "descricao": "ELETRICISTA HORA NORMAL",
  "unidade": "H",
  "grupo": "Eletromecânica"
}
```

10.3 Exemplo de resposta

```
{
  "solicitacao_id": 21,
  "resultado": "BLOQUEAR",
  "motivo": "O MOKKA aprendeu com 4 decisões: para essa descrição, use o código
7000109",
  "candidatos": [
    {
      "codigo": "7000109",
      "descricao": "ELETRICISTA - HORA NORMAL",
      "nome_padrao": "ELETRICISTA - HORA NORMAL",
      "unidade": "H",
      "categoria": "MAO DE OBRA",
      "aderencia": 100,
      "classe": "DUPLICADO",
      "diferencas": [
        "aprendido com 4 decisões"
      ]
    },
    {
      "codigo": "7000146",
      "descricao": "SV ELETRICISTA HORA EXTRA 100%",
      "nome_padrao": "ELETRICISTA - HE DOM/FER",
      "unidade": "H",
      "categoria": "MAO DE OBRA",
      "aderencia": 79,
      "classe": "DIFERENTE",
      "diferencas": [
        "100%",
        "dom/fer + hora extra",
        "hora normal"
      ]
```

```
    }
  ],
  "sugestao": null
}
```

10.4 Integração entre componentes

- **Front-end → Java:** a tela chama a API REST do Java com o token de entrada.
- **Java → Python:** o Java chama o serviço de similaridade por HTTP em 127.0.0.1:8000, com um segredo que muda a cada vez que o MOKKA liga. O Python só ouve o próprio computador e recusa quem não manda o segredo.
- **Java → PostgreSQL:** o Java grava e lê tudo no banco por JDBC. O Python não grava no banco durante o uso.
- **Python → banco, na carga:** o exportar.py monta o catálogo saneado (data.sql) a partir da planilha, e o Java carrega no banco ao ligar.
- **Se o Python cair:** o Java responde sozinho pelo banco (modo reserva). Se o Java cair, a tela continua com o catálogo guardado no navegador e envia os pedidos quando o sistema volta (modo local).

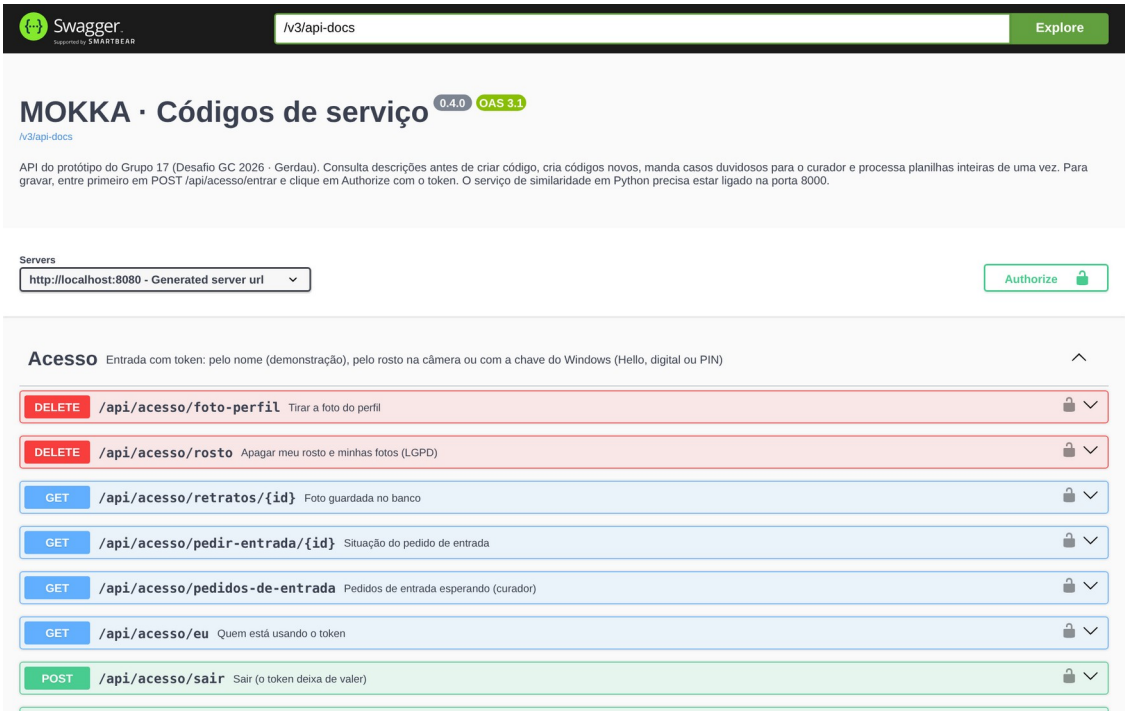


Figura 9. Swagger com as rotas da API

11. Instruções de uso

11.1 Acessar o sistema

Endereço do sistema: http://localhost:8080 no computador que liga o MOKKA, ou o link https do grupo (Tailscale) para quem foi convidado.

1. Ligar o MOKKA (MOKKA.bat, opção 1, ou o atalho "1 - Ligar o MOKKA").
2. Abrir o Edge ou o Chrome no endereço.
3. Entrar pelo rosto, pela chave do Windows ou pelo nome. Sem câmera: "Estou sem câmera" pede a liberação a um curador.

4. Na primeira vez, o botão Tutorial, no topo, mostra o que o MOKKA faz.

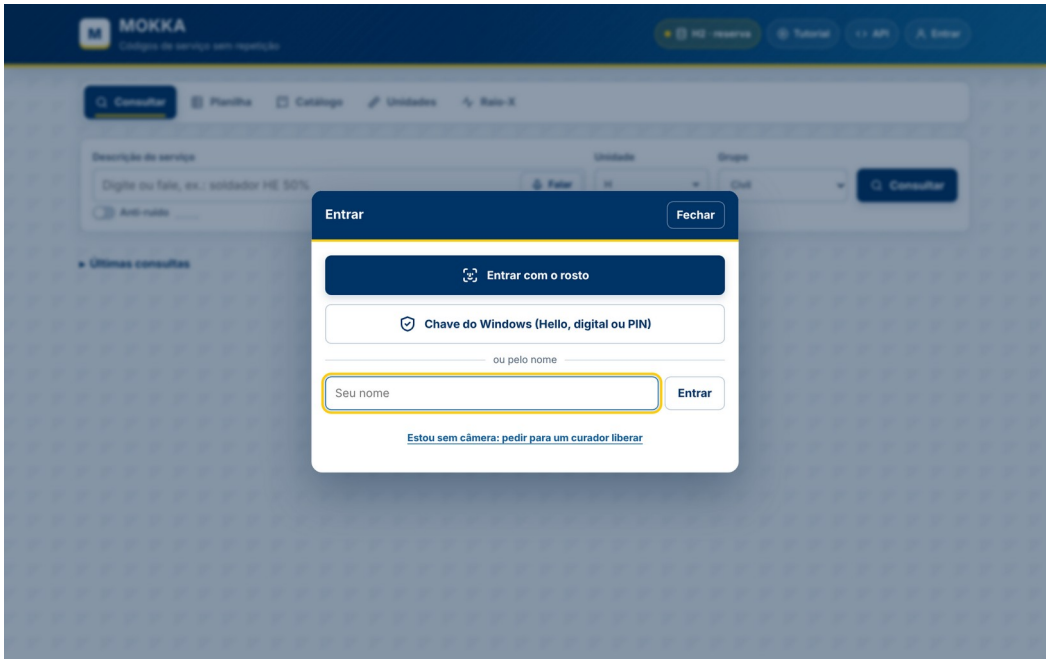


Figura 10. Entrada no sistema

11.2 Pesquisar um serviço

1. Abrir a aba Consultar.
2. Digitar a descrição ou clicar em Falar; escolher a unidade e o grupo.
3. Clicar em Consultar.
4. Ler o resultado e os parecidos (as setas mostram os próximos).

11.3 Reutilizar um código

1. Escolher o serviço adequado na lista.
2. Conferir a descrição original, o nome padrão, a unidade e o que muda.
3. Clicar em "Usar este" (ou no botão Usar logo abaixo da consulta).
4. Utilizar o código no SAP.

11.4 Solicitar novo código

1. Consultar o serviço e conferir todas as sugestões.
2. Clicar em "Criar código novo".
3. Se o MOKKA pedir, escrever por que é diferente dos que já existem.
4. Se for novo, o código sai na hora; se for parecido ou repetido, o pedido vai para o curador e aparece na fila dele.

11.5 Validar uma sugestão (curador)

1. Abrir a aba Curador.
2. Ver os pedidos iguais juntos, com o pedido e o código que já existe lado a lado e a certeza do MOKKA.
3. Aprovar (cria o código) ou recusar (usa o existente), um a um ou o grupo todo.
4. Se errar, usar Desfazer: o que o MOKKA aprendeu também é desfeito.

11.6 Analisar uma planilha

1. Abrir a aba Planilha.
2. Escolher o arquivo (Excel, CSV, texto ou JSON) ou colar a lista.
3. Clicar em Analisar e conferir cada linha.
4. Importar no banco e baixar o Excel organizado.

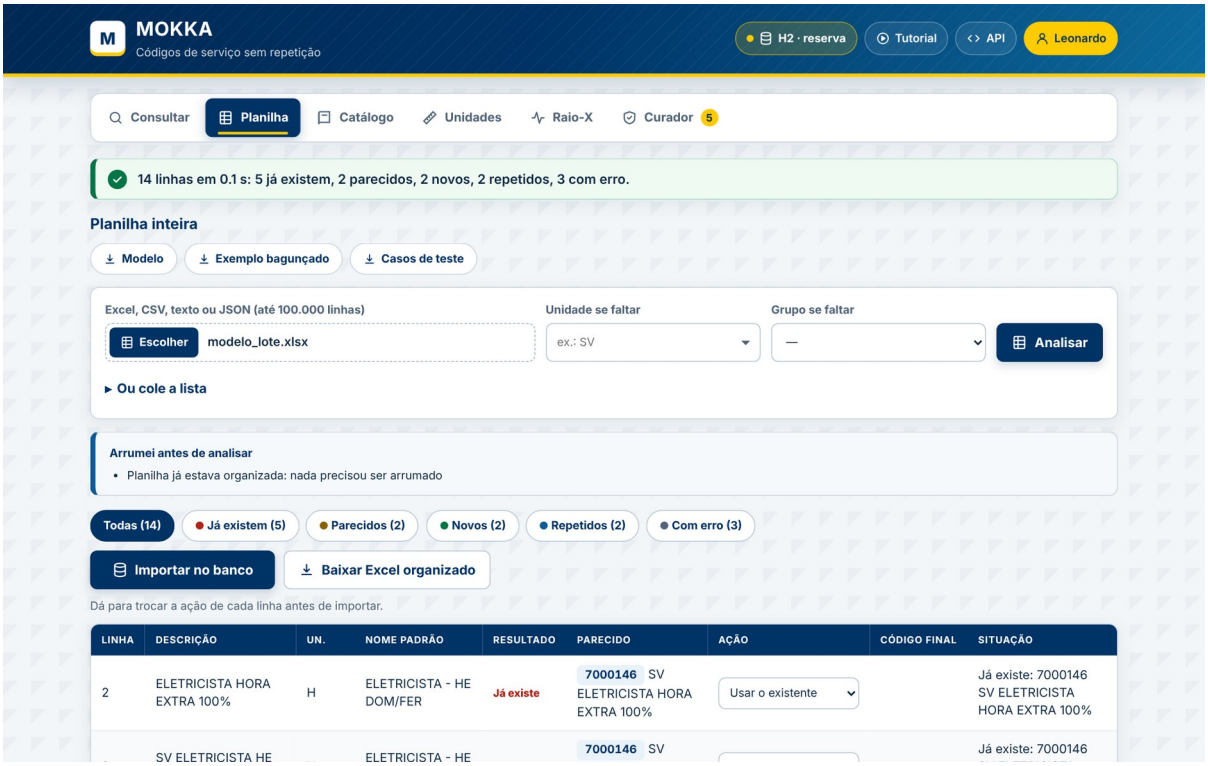


Figura 11. Planilha analisada, cada linha com o resultado

11.7 Acompanhar os resultados

A aba Raio-X, que só o curador e o administrador veem, é o painel do MOKKA: consultas por dia e por resultado, repetidos barrados, códigos novos, reuso, pedidos com o curador, regras aprendidas, quem usa e as consultas mais recentes, atualizando sozinho. Embaixo fica a saúde do catálogo, com a nota de 0 a 100 antes e depois.

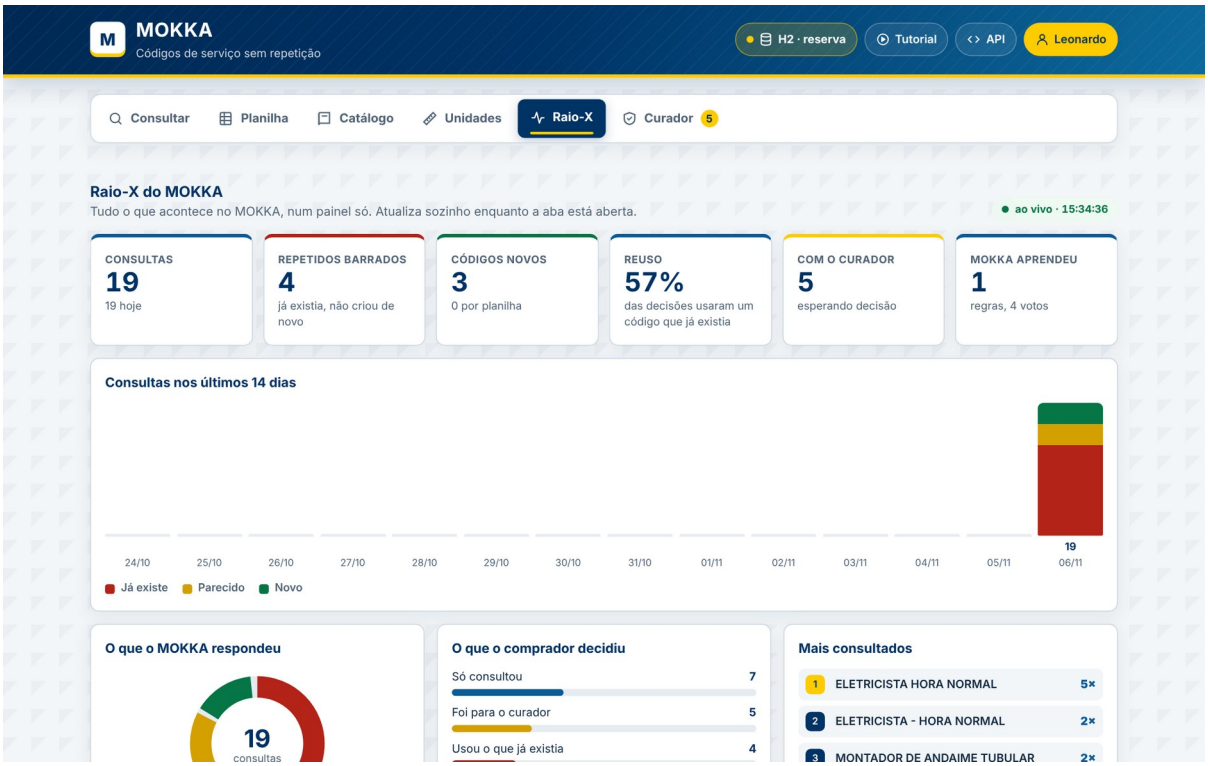


Figura 12. Raio-X: painel ao vivo

12. Instalação e configuração

O jeito mais simples é o MOKKA.bat (Windows) ou o MOKKA.command (Mac e Linux): eles fazem todos os passos abaixo sozinhos. O passo a passo à mão está em docs/COMO_ABRIR_SEM_BAT.md.

12.1 Pré-requisitos

Item	Versão
Java	17 ou mais novo (testado com o Temurin 17)
Python	3.10 a 3.13
PostgreSQL	16 (opcional; o MOKKA.bat baixa uma versão portátil). Sem ele, o H2 na memória
Navegador recomendado	Microsoft Edge ou Google Chrome (a voz não funciona no Firefox)
Outras ferramentas	internet na primeira vez (bibliotecas); Git para publicar; Tailscale para o link do grupo

12.2 Configuração do PostgreSQL

- Banco: mokka · usuário: mokka_app · servidor: 127.0.0.1 · porta: 5433.
- O MOKKA.bat cria o banco e o usuário na primeira vez e gera a senha, guardada na pasta .ferramentas do computador (fora do GitHub).
- Ligar e desligar à mão: powershell -ExecutionPolicy Bypass -File scripts\postgres.ps1 ligar (ou parar).
- As tabelas são criadas pelo schema.sql toda vez que o Java liga.

12.3 Configuração do Python

```
cd MOKKA\saneamento
python -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install -r requirements.txt
python exportar.py dados\base_codigos_servico.xlsx --tudo
$env:MOKKA_SEGREDO = "troque-por-um-segredo"
python api.py dados\base_codigos_servico.xlsx
```

12.4 Configuração do Java

```
cd MOKKA\codigos-api
.\mvnw.cmd -q -DskipTests package
$env:MOKKA_SEGREDO = "o mesmo do Python"
java -jar target\codigos-api-0.4.0.jar --mokka.dados=../saneamento/saida/data.sql

Com o PostgreSQL: defina DB_URL, DB_USER e DB_PASSWORD e acrescente --
spring.profiles.active=postgres
```

12.5 Configuração do front-end

Não precisa instalar nada: o Java serve a tela em http://localhost:8080. Para editar a tela sem compilar, o MOKKA.bat usa a pasta front, e basta apertar F5 no navegador depois de salvar.

12.6 Variáveis de ambiente

Variável	Finalidade	Valor de exemplo
MOKKA_SEGREDO	segredo entre o Java e o Python (o MOKKA.bat gera um novo a cada vez)	um texto aleatório
DB_URL	endereço do PostgreSQL	jdbc:postgresql:// 127.0.0.1:5433/mokka
DB_USER	usuário do banco	mokka_app
DB_PASSWORD	senha do banco (não versionar)	(fica em .ferramentas)
SIMILARIDADE_URL	endereço do serviço Python	http://127.0.0.1:8000
MOKKA_ADMINS	nomes que entram como administrador ou curador	Leonardo Lopes,Thales,Thirza,Bárbara,Lukas
MOKKA_DADOS	catálogo saneado carregado ao ligar	../saneamento/saida/data.sql
MOKKA_CHAVE_FOTOS	arquivo da chave que criptografa as fotos	../.ferramentas/chave-fotos.key
MOKKA_SO_CHAVE	aceitar só rosto e chave do Windows, sem entrada pelo nome	false

13. Critérios de aceite

13.1 Dados

Critério	Situação
Uma amostra real está armazenada no PostgreSQL	atendido: a base inteira (7.937 códigos)
Os dados brutos são preservados	atendido: tabela base_original

Critério	Situação
A coluna D pode ser padronizada	atendido: descricao_limpa e descricao_padrao
A coluna C pode ser consultada	atendido: busca e ficha do código
Os registros tratados podem ser relacionados aos originais	atendido: codigo_canonico e de-para

13.2 Busca

Critério	Situação
O usuário consegue pesquisar uma descrição	atendido, por texto e por voz
O sistema encontra descrições semelhantes	atendido: até 10 candidatos, de 5 em 5
Os candidatos são apresentados em ordem compreensível	atendido: classe, aderência e o que muda
O código do serviço aparece nos resultados	atendido
O sistema informa quando não há correspondência	atendido: "Nenhum código igual" (LIBERAR)

13.3 Fluxo de negócio

Critério	Situação
O usuário consegue reutilizar um código	atendido: Usar este
O usuário consegue confirmar uma sugestão	atendido
O usuário consegue solicitar um novo código	atendido
A solicitação possui justificativa	atendido: obrigatória quando há parecido
A solicitação pode ser aprovada ou rejeitada	atendido: curador, com Desfazer
As decisões ficam registradas	atendido: solicitacao, auditoria e aprendizado

13.4 Integração

Critério	Situação
Python, Java, PostgreSQL e front-end integrados	atendido: teste geral de ponta a ponta 35 de 35
Os principais erros são tratados	atendido: seção 14
A demonstração funciona de ponta a ponta	atendido: roteiro do pitch
Existe documentação suficiente para compreender e usar	atendido: este documento, README e docs/

14. Tratamento de erros

Mensagens reais exibidas pelo sistema:

Situação	Mensagem exibida	Ação recomendada
Registro incompleto	Informe a descrição	preencher a descrição
Unidade desconhecida	Unidade não conhecida: caixa. Veja a aba Unidades.	escolher uma unidade da lista
Código inexistente	Código não encontrado: 1234567	conferir o código ou buscar pela descrição

Situação	Mensagem exibida	Ação recomendada
Descrição maior que 40 caracteres (pela API)	A descrição deve ter até 40 caracteres	na tela, o MOKKA abrevia sozinho ao consultar
Possível duplicidade	Já existe: 7000109 ELETRICISTA - HORA NORMAL	usar o código existente ou justificar
Ausência de resultados	Nenhum código igual; os parecidos são diferentes	criar o código novo
Arquivo em formato errado	Formato .pdf não aceito. Envie Excel (.xlsx), CSV, texto (.txt) ou JSON, ou cole a lista.	salvar em um dos formatos aceitos
Arquivo grande demais	Arquivo maior que 100 MB. Divida em partes.	dividir a planilha
Sem entrar	Entre no sistema para continuar.	entrar de novo
Muitas tentativas de entrada	bloqueio temporário (erro 429)	esperar ou pedir ao administrador para desbloquear
Falha de conexão com o banco	Falha ao acessar o banco de dados. Tente de novo; se continuar, reinicie o sistema.	reiniciar pelo MOKKA.bat
Python indisponível	aviso "modo reserva": a consulta segue pelo banco	reiniciar o MOKKA quando puder
Java indisponível	faixa amarela "Sem conexão": a tela segue com o catálogo guardado	ligar o MOKKA; os pedidos são enviados sozinhos
Endereço errado na API	Endereço não existe. Veja a lista em /swagger.	conferir a rota no Swagger

15. Limitações

15.1 Limitações dos dados

A qualidade das sugestões depende da qualidade dos dados originais. 247 descrições estavam cortadas em 40 caracteres e ficaram separadas por segurança; os códigos-sigla (como "SV-MEC-01-HN") precisam de uma tabela de siglas da Gerdau.

15.2 Limitações da similaridade

Uma descrição semelhante não garante que dois serviços sejam iguais. A régua só junta quando cargo, tipo de hora, percentual, medida e unidade batem; nos casos duvidosos, a validação humana (curador) continua necessária.

15.3 Limitações da automação

A automação pode não reconhecer todos os formatos e abreviações. Cinco perguntas de negócio dependem da Gerdau e juntariam mais códigos: EVENTUAL muda o serviço? (143 códigos), À DISPOSIÇÃO (129), ADM (72), H e SV são a mesma unidade para mão de obra? (63), ESP (42). A mudança é uma linha de configuração no deduplicar.py.

15.4 Limitações da classificação

HH, SV e H só são sugeridos automaticamente quando há regra ou correspondência confiável. Unidade diferente nunca é tratada como o mesmo serviço.

15.5 Limitações de integração

Front-end, Java, Python e PostgreSQL estão integrados e funcionam juntos, sem passo manual. O Python é a única parte que pode ficar fora sem parar o sistema (modo reserva). Ainda não há integração direta com o SAP: o MOKKA entrega o Excel "Pronto para o SAP".

15.6 Limitações de acesso

Existe autenticação (rosto, chave do Windows ou nome), token de 8 horas, três perfis (COMPRADOR, CURADOR e ADMIN), limite de tentativas e auditoria. A entrada pelo nome serve para a demonstração; em produção, o certo é o login corporativo da Gerdau (AD ou SSO).

15.7 Limitações de escala

Testado com a base inteira (7.937 códigos), consultas em menos de 10 ms e lotes de até 100.000 linhas (cerca de 3 minutos no pior caso). Roda no computador de quem liga; para muitos usuários ao mesmo tempo, o próximo passo é um servidor.

16. Segurança e preservação dos dados

- **Preserva a base original:** a planilha fica intacta em base_original; nenhum código é apagado (o repetido fica BLOQUEADO e aponta para o oficial).
- **Não apaga sem confirmação:** zerar o banco pede confirmação e só o administrador pode.
- **Mantém histórico:** auditoria de quem fez o quê, quando e de onde.
- **Controla quem aprova:** só CURADOR e ADMIN decidem pedidos e veem o Raio-X; o comprador recebe 403.
- **Fotos protegidas:** criptografadas, e cada pessoa pode apagar o próprio rosto (LGPD).
- **Serviços fechados:** o Python e o PostgreSQL só atendem o próprio computador; o Python exige o segredo.
- **Navegador:** cabeçalhos de proteção (CSP, anti-iframe, sem cache na API) e nada abre sem entrar.
- **Link do grupo:** rede privada do Tailscale; pelo link, o Swagger e o console do banco ficam fechados.
- **Dados fora do GitHub:** a planilha, as senhas e a chave das fotos nunca vão para o repositório.

17. Testes

17.1 Testes de dados

Teste	Resultado
Descrições com abreviações	dicionário de 136 termos; testes automáticos do Python 9 de 9
Descrições repetidas	883 repetidos encontrados; repetidos dentro da própria planilha também
Descrições semelhantes	430 pares conferidos à mão: 0 serviços diferentes juntados; 145 pares com 80%+ de texto igual, todos separados
Erros de digitação e de fala	89% resolvidos (296 de 334); a V1 resolvia 0
Descrições maiores que 40 caracteres	abreviadas no padrão do SAP

Teste	Resultado
Serviços novos	15 de 15 liberados, sem sugestão falsa

17.2 Testes da API

O teste geral (MOKKA.bat, opção 2) liga o sistema e passa pelo fluxo inteiro pela API. Resultado: 35 de 35. Entre os testes:

- Consultar um serviço existente (BLOQUEAR) e um inexistente (LIBERAR).
- Correção de fala (SOLDA DOR → SOLDADOR).
- Solicitar novo código, aprovar, recusar e desfazer no curador.
- Enviar dados inválidos (unidade errada, injeção, JSON torto): nenhum erro 500.
- Testar falha de conexão: Python desligado, a consulta responde pelo banco.
- Segurança: nada abre sem token, perfis (o comprador recebe 403 no curador e no Raio-X), força bruta, cabeçalhos, link do grupo.
- Planilha em lote, em JSON e em texto; Excel organizado.
- Raio-X, painel do Raio-X e Tutorial.

17.3 Testes do front-end

- Abrir o Tutorial e a entrada.
- Realizar uma busca por texto e por voz.
- Visualizar sugestões e paginar os parecidos.
- Confirmar uma sugestão.
- Solicitar novo código.
- Visualizar mensagens de erro.
- Testar no computador e no celular (a tela se adapta).

Evidências: o testar.log de cada execução fica na pasta 21 - Integração completa; as validações do Python saem em saneamento/saida.

18. Operação e manutenção

A base deve ser revisada periodicamente para:

- Incluir novas abreviações (saneamento/docs/dicionario.csv).
- Corrigir regras de padronização (padronizar.py e regua.py).
- Analisar novas duplicidades (deduplicar.py e o Raio-X).
- Revisar pendências e pedidos do curador.
- Atualizar o catálogo (o MOKKA monta de novo a partir da planilha ao ligar).
- Avaliar solicitações de novos códigos.
- Verificar a qualidade das sugestões (python validar.py e o teste geral).
- Manter os backups atualizados (a pasta do PostgreSQL fica em .ferramentas).

19. Solução de problemas

Problema	Possível causa	Ação recomendada
Sistema não abre	MOKKA desligado ou Java sem compilar	rodar MOKKA.bat, opção 1; esperar "PRONTO"
Busca não retorna dados	catálogo não carregou ou filtro de grupo	conferir o selo do banco no topo; limpar o grupo
API indisponível	Java parado ou porta 8080 ocupada	MOKKA.bat parar e ligar de novo
Banco não conecta	PostgreSQL desligado	MOKKA.bat liga sozinho; ou scripts\ postgres.ps1 ligar; sem ele, usa o H2
Sugestão incorreta	abreviação nova ou regra de negócio	curador recusa ou aprova (o MOKKA aprende); incluir o termo no dicionário
Registro duplicado	código criado antes do MOKKA	o Raio-X mostra as famílias; o deduplicar.py junta na próxima carga
Voz não funciona	navegador sem suporte ou sem internet	usar Edge ou Chrome; digitar
Tela antiga depois de atualizar	cache do navegador	apertar F5 (o MOKKA já pede ao navegador para conferir)

20. Evidências visuais

Evidência	Onde está
Tela inicial (Tutorial)	Figura 13
Tela de consulta	Figuras 2 e 4
Resultados e sugestões	Figuras 4 e 5
Tela de validação (curador)	Figura 7
Solicitação de novo código	Figura 6
Catálogo	Figura 14
Banco e modelo de dados	Figura 3
Diagrama da arquitetura	Figura 1
Diagrama do fluxo do usuário	Figura 8
Integração entre front-end e back-end	Figuras 9 e 12
Evidência dos testes	seção 17 e testar.log

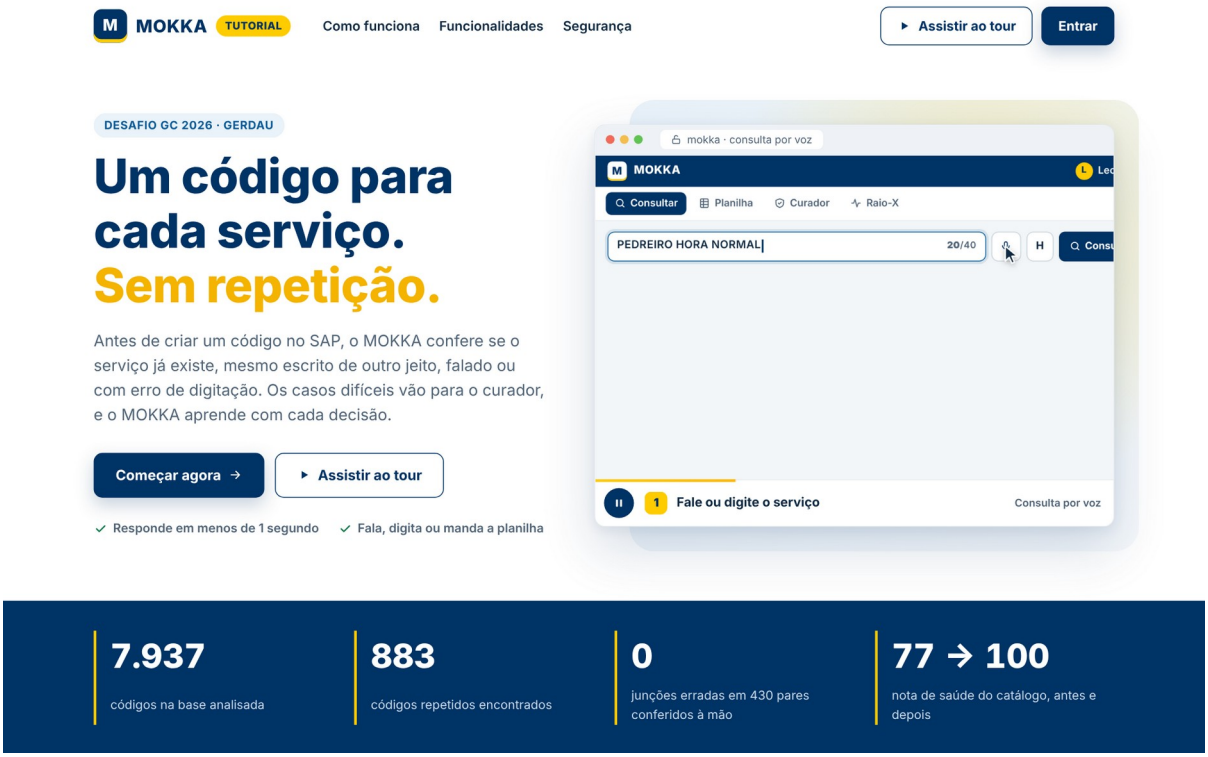


Figura 13. Tutorial com a demonstração animada

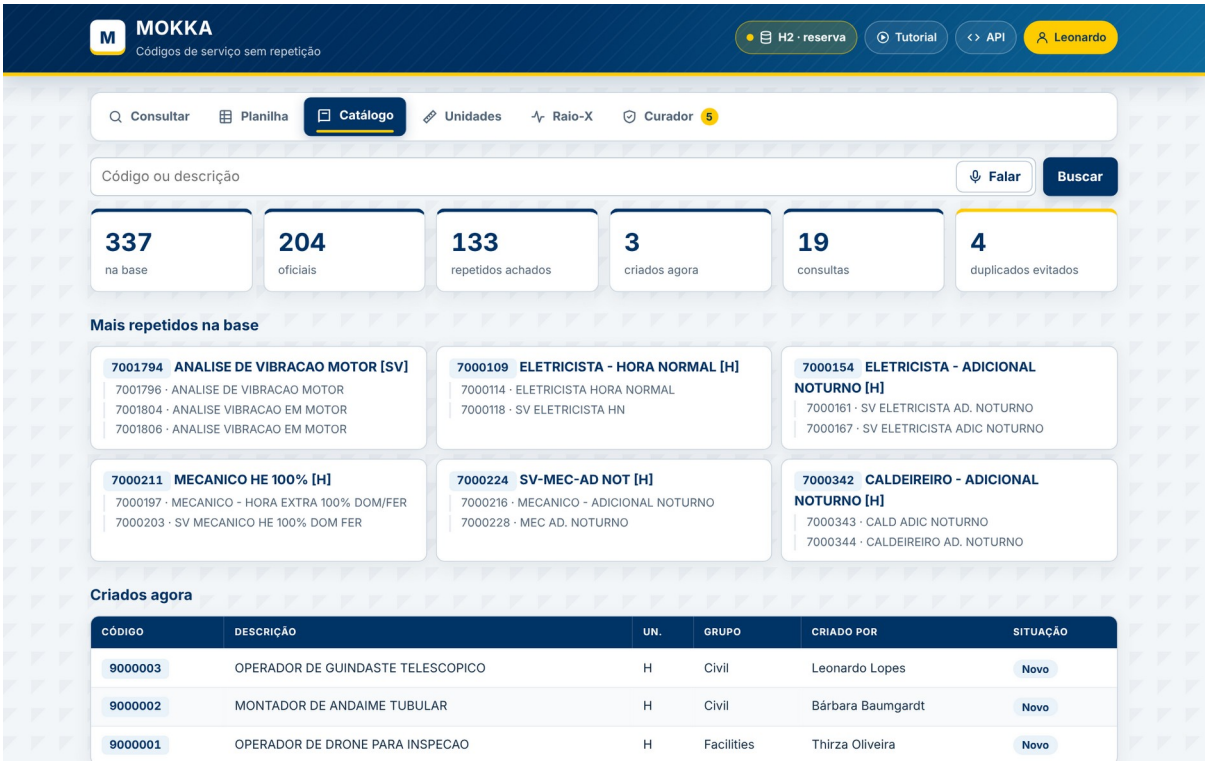


Figura 14. Catálogo: oficiais e repetidos



Figura 15. Raio-X: saúde do catálogo

As telas foram capturadas com a base de exemplo (fictícia) do repositório.

21. Próximas evoluções

Evolução	Situação
Painel de indicadores	feito: Raio-X ao vivo
Autenticação de usuários	feito: rosto, chave do Windows e nome, com perfis; falta o login corporativo
Diferentes níveis de aprovação	feito: comprador, curador e administrador; dá para acrescentar aprovação por grupo
Integração com o SAP	próximo passo: criar o código direto no SAP por API depois do BLOQUEAR, REVISAR ou LIBERAR
Servidor	próximo passo: servidor da Gerdau com HTTPS e o login corporativo
Busca mais inteligente	próximo passo: usar a descrição longa do SAP para os textos cortados e uma tabela de siglas
Regras de negócio da Gerdau	responder as 5 perguntas da seção 15.3